```
Output:
object(DateTimeZone)#2 (2) {
  ["timezone_type"]=>
  int(2)
  ["timezone"]=>
  string(3) "PDT"
}
```

... alternatively `DateTimeZone` instances can be supplied as a constructor argument to `new DateTime()`, or set after construction via `DateTime::setTimezone()`. `DateTimeZone` objects themselves accept a string argument, which is either a UTC time offset, a time zone abbreviation such as "GMT," or a canonical `tzdata` name. We could use `PDT` or `America/Los_Angeles` to build a time zone object with the same internal value; let's go with the canonical `tz database` name:

```
$tz = new DateTimeZone('America/Los_Angeles');
$dt = new DateTime('Tuesday, 03-Sep-2024 05:00:00', $tz);
print $dt->format('Y-m-d H:i:s T');
```

```
Output:
2024-09-03 05:00:00 PDT
```

Note that `DateTime` objects can have a `DateTimeZone` instance as a second argument only if the first argument is not a Unix timestamp or contains any time zone information:

```
$dt = new DateTime('@1725148800', $tz);
print $dt->format('Y-m-d H:i:s T') . PHP_EOL;

$dt = new DateTime('Tuesday, 03-Sep-2024 05:00:00 CET', $tz);
print $dt->format('Y-m-d H:i:s T');
```

```
Output:
2024-09-01 00:00:00 GMT+0000
2024-09-03 05:00:00 CET
```

In both cases above, the time zone argument is ignored. The system time zone (which happens to be GMT on the system I ran this code on) is used in the first example (a Unix timestamp), and the time zone specified in the input string (CET) is used in the second. Things get a little tricky here: PHP will have a default timezone set via the